

## Chapter 13

# 現代網頁製作

Horazon

互動媒體設計

# 前端開發的演進

傳統網頁使用 HTML、CSS、JavaScript 分開撰寫（前端三劍客）。

隨著 Facebook、Gmail 等複雜應用出現，傳統寫法難以維護。

**現代前端開發** 引入了框架、工具鏈與型別系統，讓大型專案的建立與維護變得可行。

# TypeScript 簡介

TypeScript (TS) 是 JavaScript 的超集，由微軟開發。

- **型別檢查**：宣告變數型別，寫錯時編輯器立即報錯，不必等到上線才發現
- **更聰明的自動完成**：VS Code 能準確提示物件屬性與方法
- **漸進式導入**：TS 完全相容 JS，可逐步遷移，不需一次重寫

業界新專案幾乎預設使用 TypeScript。

# 常見的前端框架

前端框架幫我們解決三件事：**管理狀態**（資料變了，畫面自動更新）、**元件化**（把導覽列、卡片、按鈕做成可重複使用的積木）、**維護性**（大型專案不再一團混亂）。

| 框架      | 開發者    | 特色            |
|---------|--------|---------------|
| React   | Meta   | 市佔最高，生態最完整    |
| Vue.js  | 社群     | 學習曲線平緩，亞洲廣泛使用 |
| Angular | Google | 適合大型企業級專案     |

# 後端是什麼？

後端是使用者「看不到的部分」，負責處理資料與業務邏輯：

- **資料庫**：儲存帳號、文章、訂單等資料（MySQL、PostgreSQL、MongoDB）
- **API**：前端透過 API 向後端請求資料，後端回傳 JSON
- **驗證與安全**：登入、權限控管、保護敏感資料

常見的後端語言與框架：

| 語言         | 框架                | 特色         |
|------------|-------------------|------------|
| JavaScript | Node.js / Express | 前後端同一語言    |
| Python     | Django / FastAPI  | 簡潔、AI 生態豐富 |
| PHP        | Laravel           | 傳統 CMS 主力  |

# 前端 → 後端 → 全端

## 前端框架 (React / Vue / Angular)

管理畫面、狀態與元件 — 使用者看得到的部分

前後端透過 **API** 交換資料 (前端送出請求，後端回傳 JSON)

## 後端 (Node.js / Python / PHP)

處理資料庫、登入驗證與業務邏輯 — 使用者看不到的部分

兩者整合在同一個專案..

## 全端框架 (Next.js / Nuxt.js)

前後端共用一套程式碼，開發與部署都更簡單

# 全端框架的崛起

基於三大框架，衍生出「全端框架」，前後端可在同一專案中開發：

- Next.js（基於 React）：目前業界最主流的全端框架
- Nuxt.js（基於 Vue）：Vue 生態的全端解決方案
- SvelteKit（基於 Svelte）：效能導向的新興選擇

這些框架已成為現代 Web 開發的**業界標準**。

# AI 世代的網頁開發

現代框架學習曲線雖然陡峭，但開發模式已徹底改變。

**AI 成為最強大的寫程式夥伴 (Pair Programmer)：**

- 不需從零死背語法
- 只需具備**清晰的邏輯與規劃能力**
- AI 自動生成程式碼，你負責審查與引導

目前 AI 輔助開發主要分為：**雲端生成平台** 與 **本機 Agent AI**。



# AI 輔助開發工具概覽

| 類型          | 代表工具                   | 運作方式       |
|-------------|------------------------|------------|
| 通用語言模型      | ChatGPT、Claude         | 對話、除錯、解釋   |
| AI 程式碼編輯器   | Cursor、Copilot         | 編輯器內即時補完   |
| 雲端生成平台      | Bolt.new、v0.dev        | 瀏覽器中生成完整專案 |
| 本機 Agent AI | Claude Code、Gemini CLI | 操作本機檔案與終端機 |

實際開發時，**混合使用**多種工具效果最佳。

# 通用型語言模型

ChatGPT 與 Claude 是目前最廣泛使用的 AI 對話工具。

## 在網頁開發上的應用

- 除錯：貼上錯誤程式碼，請 AI 分析原因
- 功能撰寫：「幫我寫一個 JavaScript 輪播圖元件」
- 程式碼解釋：用白話說明看不懂的程式碼
- 重構建議：找出可改進之處

提示詞越精確，輸出品質越高。

# WebContainers：瀏覽器裡的開發環境

過去執行 React 等現代框架，必須在本機安裝 Node.js、設定環境、安裝套件，過程繁瑣。

WebContainers 技術讓我們可以**直接在瀏覽器**中運行完整的 Node.js 環境。

搭配 AI，只需在網頁分頁中輸入需求，即可：

- 瞬間搭建現代前端專案並即時預覽
- **完全不需在本地安裝任何軟體**

這是 Bolt.new 等雲端生成平台的核心基礎。

Bolt.new (StackBlitz) 是目前最受歡迎的 AI 雲端生成平台。

- **自然語言即生成**：「幫我做一個待辦事項 App」，數秒得到完整 React 專案
- **瀏覽器即時預覽**：基於 WebContainers，無需安裝軟體
- **持續對話修改**：像和工程師溝通一樣繼續調整
- **一鍵部署**：直接部署取得公開網址

網址：[bolt.new](https://bolt.new)

v0.dev (Vercel) 專注於 **UI 元件的生成**。

- **元件導向**：生成按鈕、表單、卡片、儀表板等精美元件
- **shadcn/ui 整合**：產出的程式碼符合業界標準元件庫
- **直接複製貼上**：可無縫整合進 Next.js 或 React 專案
- **視覺化即時預覽**：馬上看到元件外觀

## 最適合

已有專案骨架，需要快速補充**漂亮且實用的 UI 元件**時使用。

網址：[v0.dev](https://v0.dev)

Lovable（前身 GPT Engineer）是 AI 全端開發平台。

- **完整應用程式生成**：UI、後端邏輯、資料庫結構一次生成
- **Supabase 整合**：直接連結資料庫，產出有真實資料的應用
- **GitHub 同步**：專案同步到 GitHub，方便後續本機開發

## 適合場景

想做出**有資料庫的 MVP（最小可行產品）** 時使用。

網址：[lovable.dev](https://lovable.dev)

# 雲端生成平台比較

| 工具              | 強項          | 最適合情境           |
|-----------------|-------------|-----------------|
| Bolt.new        | 快速搭建前端專案    | 原型、Landing Page |
| v0.dev          | 高品質 UI 元件   | 補充現有專案元件        |
| Lovable         | 全端 + 資料庫    | MVP、後端需求        |
| Replit Agent    | 多語言後端       | 後端 API          |
| Firebase Studio | Google 服務整合 | 需 Google 雲端功能   |

這些平台都有免費方案，付費通常以訊息數或計算資源計費。

# Agent AI：本機開發的革命

雲端平台方便，但最終專案還是需要回到本機繼續開發。

**Agent AI 能自主執行多步驟任務**，不只回答問題，還能：

- 讀寫本機檔案、建立與修改程式碼
- 執行終端機指令（安裝套件、執行測試）
- 搜尋理解整個程式碼庫後再下手修改
- 發現錯誤後自動修正，直到成功

只需告訴它「我要做什麼」，它自己拆解步驟並完成。



# 主流 Agent AI 工具

| 工具                   | 運作環境    | 特色                   |
|----------------------|---------|----------------------|
| Cursor Agent         | 編輯器     | 理解整個專案、同時修改多個檔案      |
| GitHub Copilot Agent | VS Code | 多模型選擇、GitHub 深度整合    |
| Claude Code          | 終端機     | 擅長大型程式碼庫重構           |
| Gemini CLI           | 終端機     | 免費、100 萬 token 超長上下文 |

CLI = Command Line Interface，在終端機黑畫面操作。

# Agent AI 的典型工作流程

1. **描述需求**：「在 React 專案新增登入頁，使用 Tailwind CSS」
2. **Agent 分析專案**：自動讀取相關檔案，理解現有架構
3. **執行多步驟修改**：建立 LoginPage.jsx → 修改 router.js → 更新 App.jsx
4. **回報結果**：若有錯誤，自動嘗試修正
5. **你審查並繼續**：確認修改內容，再交代下一個任務

# AI 輔助開發的最佳實踐

## 應該做的

- 每次任務範圍要小：「新增搜尋功能」比「做完整電商」更容易成功
- 使用版本控制 (Git)：AI 修改前先 commit，出錯可以還原
- 審查每一行程式碼：你要對最終結果負責

## 避免的陷阱

- 盲目複製貼上：不理解程式碼，出 Bug 時無法修復
- 描述模糊：「幫我做好一點」這種 Prompt 沒有意義

# AI 開發者的新角色

在 AI 崛起的時代，工程師的工作**提升到更高的層次**。

**從「寫程式的人」→「指揮程式的人」**

- 以前：80% 時間寫程式，20% 時間思考架構
- 現在：20% 時間引導 AI，80% 時間思考**產品邏輯與使用者體驗**

**核心競爭力不再是「記得多少語法」，而是「能不能把模糊的需求，拆解成 AI 能執行的清晰指令」。**

# 本章重點回顧

- **TypeScript**：JavaScript 的超集，加入型別系統，業界標準
- **前端框架**：React / Vue / Angular，衍生 Next.js 等全端框架
- **雲端平台**：Bolt.new、v0.dev、Lovable — 瀏覽器即可生成專案
- **Agent AI**：Cursor、Copilot、Claude Code、Gemini CLI — 自主多步驟執行
- **AI 開發核心**：清晰的邏輯規劃 > 語法記憶